Software Engineering Lab

Lab 4: VibeCoding

Maze Chase (pygame) - Fix & Feature Additions

Name	Vaishnavi Goyal
SRN	PES2UG24CS571
Section	5J
Course / Lab	SE Lab 4 - VibeCoding
Date	15/10/2026
Vibe coding tool used	Claude

1. Repository Links

Assigned repo (source)	https://github.com/SETAPESU26/11_maze_chase
My submission repo	https://github.com/whyshoee/SELABS
Lab 4 folder link	https://github.com/whyshoee/SELABS/tree/main/Lab-4
Chat link (if shareable)	https://claude.ai/share/fc09bc25-b854-47b8-8679-930d1c4e51e2

2. Before Changes

2.1 Video (10 seconds, before any changes)

File name: before.mp4 - uploaded in Lab-4/

2.2 Screenshots from the 'before' video



2.3 Observations

Running python main.py on the starter code printed a pkg_resources deprecation UserWarning and the pygame banner, then the game opened: a blue player (top-left), a single red enemy chasing with BFS, and a green EXIT square in the middle of the maze. The main problem was that the player could move over the maze lines and was not confined within the walls. Reading the code through the LLM also turned up smaller issues (HUD/EXIT text overflow, a caught-and-won frame clash, and a per-frame import), listed below.

3. Bugs Found and Fixes

Record each bug in the starter code and how it was fixed through prompting. All of these came from the first prompt (pasting the code, the warning and the symptom). game/maze.py was checked and left unchanged.

#	Bug / Symptom	File & Cause	Prompt attempt #	Fixed?
1	Player passes through the maze walls and is not confined within the maze lines	game/entities.py - Player._valid() only checked that the player stayed inside the window; it never tested wall positions	1	Yes - builds thin wall rects (3 px, matching the drawn lines) for each cell the player overlaps and rejects moves that collide
2	pkg_resources deprecation UserWarning and pygame greeting printed at startup	main.py - warning comes from pygame's own import (not a game-logic bug)	1	Yes - warnings filter and PYGAME_HIDE_SUPPORT_PROMPT set before pygame is imported
3	HUD hint text cut off at the right edge	game/game_engine.py - 22pt font is wider than the 572 px window	1	Yes - smaller hud_font
4	EXIT label overflows its box	game/game_engine.py - 22pt label wider than the 34 px exit box	1	Yes - small font, centered in the box
5	Caught and won on the same frame could draw two overlays	game/game_engine.py - update() used two independent if statements	1	Yes - if/elif, being caught takes priority
6	bfs re-imported inside Enemy.update on every frame	game/entities.py - import placed inside the method	1	Yes - import moved to top of the file

4. Feature Tasks (Prompts and Results)

Lab goal: complete each task in 3-4 prompt attempts. Fill in the actual prompts you used.

Task 1: Multiple Enemies

Requirement: Add 2 more enemies from different corners; each chases independently (self.enemies list).

Attempt	Prompt used (paste exactly)	Result
1	In my maze-chase pygame game, I have one Enemy that uses BFS to chase the player. Add two more Enemy instances starting at different	Done in 1 prompt. self.enemies holds 3

Attempt	Prompt used (paste exactly)	Result
	maze corners. Each should independently call bfs() on every update. List them in a self.enemies list and loop over them. Give me the complete updated files.	enemies (red bottom-right, orange top-right, purple bottom-left); each has its own timer and calls bfs() independently. Visible in the after screenshots.

Files changed: game/game_engine.py, game/entities.py (optional color argument on Enemy)

Task 2: Speed Up Over Time

Requirement: Every 15 s the enemy move_interval decreases by 2 (min 5); speed tier shown in HUD.

Attempt	Prompt used (paste exactly)	Result
1	Add a difficulty ramp to my maze-chase game. Track elapsed time with pygame.time.get_ticks(). Every 15 seconds, reduce enemy.move_interval by 2 (minimum 5) for all enemies. Show current enemy speed tier in the HUD. Give me the complete updated files.	Done in 1 prompt. Elapsed time from get_ticks(); every 15 s each enemy's move_interval drops by 2 (20, 18, 16 ... 6, then 5 = cap). HUD shows elapsed time, 'Speed tier N' and the current interval, with MAX once capped. Clock and tier restart on R and freeze on the end screen.

Files changed: game/game_engine.py

Task 3: Power Pellet (Freeze)

Requirement: Yellow pellet in maze; collecting it freezes enemies for 5 s (300 frames); frozen indicator shown.

Attempt	Prompt used (paste exactly)	Result
1	Add a power pellet to my maze-chase game: a yellow circle somewhere in the maze. When the player rect collides with it, set frozen = True on all enemies and start a 300-frame countdown. While frozen, enemy.update() does nothing. Show a frozen indicator on the enemies. Give me the complete updated files.	Done in 1 prompt. Enemy has a frozen flag and update() returns early while it is set. One yellow pellet is placed in a random cell (never a corner or the exit cell); touching it freezes all enemies for 300 frames (5 s). Frozen enemies are tinted ice-blue with a light outline, and the HUD shows 'FROZEN Ns'.

Files changed: game/game_engine.py, game/entities.py (frozen flag and frozen drawing)

Task 4: Score by Distance

Requirement: Survival score increments per frame; HUD shows "Survived: Xs"; final score on game-over overlay.

Attempt	Prompt used (paste exactly)	Result
1	Add a survival score to maze-chase. Increment self.score by 1 each frame the player is alive. Display it in the HUD as "Survived: Xs" where X is score // 60. On game over, show the final score on the overlay. Give me the complete updated files.	Done in 1 prompt. Score +1 per frame the player is alive (the ending frame is not counted); HUD gets a third line 'Survived: Xs' (HUD_H raised to 66); CAUGHT!/ESCAPED! overlays show 'Survived: Xs (score N)'; score resets on R.

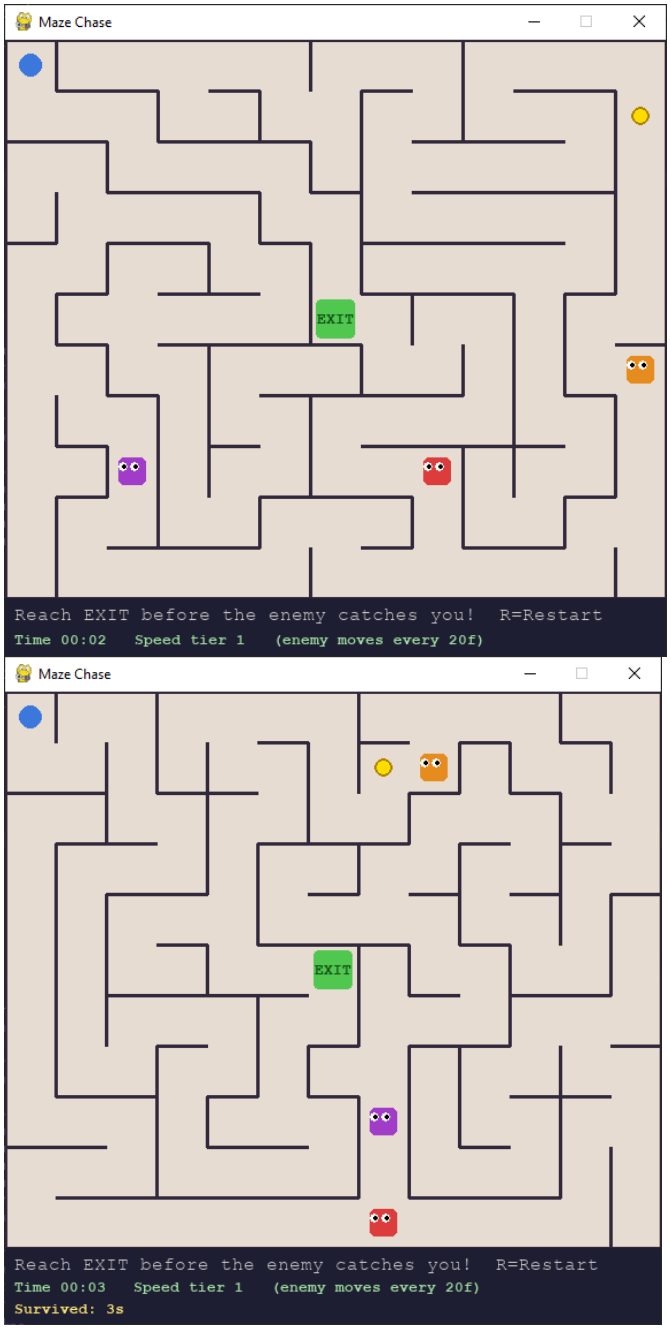
Files changed: game/game_engine.py

5. After Changes

5.1 Video (10 seconds, after all changes)

5.2 Screenshots from the 'after' video





5.3 Submission checklist (from README)

- ☑ All 4 tasks completed
- ☑ Multiple enemies work independently
- ☑ Power pellet freezes enemy correctly
- ☑ Speed ramp increases difficulty over time
- ☑ Code reviewed with LLM (chat link included)

6. Updated Code

Paste the final version of every file. Use Courier New / monospace so formatting is preserved.

main.py

```
import os
import warnings


# Silence pygame's internal pkg_resources deprecation warning and greeting.
# These must be set BEFORE pygame is imported (GameEngine imports it).
os.environ.setdefault("PYGAME_HIDE_SUPPORT_PROMPT", "1")
warnings.filterwarnings("ignore", message="pkg_resources is deprecated")


from game.game_engine import GameEngine


if __name__ == "__main__":
    engine = GameEngine()
    engine.run()
```

game/__init__.py

(empty file - no contents; it only marks the game/ folder as a Python package)

game/game_engine.py

```
import random
import pygame
from game.maze import generate_maze, CELL
from game.entities import Player, Enemy


COLS, ROWS = 13, 11
HUD_H = 66
WIDTH = COLS * CELL
HEIGHT = ROWS * CELL + HUD_H
FPS = 60


# Difficulty ramp settings
RAMP_INTERVAL_MS = 15000   # how often enemies speed up
RAMP_STEP = 2              # frames removed from move_interval each step
MIN_MOVE_INTERVAL = 5      # enemies never move faster than this


# Power pellet settings
FREEZE_FRAMES = 300        # how long enemies stay frozen after pickup
PELLET_RADIUS = 8
PELLET_COLOR = (255, 220, 0)


class GameEngine:
    def __init__(self):
        pygame.init()
        self.screen = pygame.display.set_mode((WIDTH, HEIGHT))
        pygame.display.set_caption("Maze Chase")
        self.clock = pygame.time.Clock()
        self.font = pygame.font.SysFont("monospace", 22)
        self.hud_font = pygame.font.SysFont("monospace", 16)
        self.small_font = pygame.font.SysFont("monospace", 13, bold=True)
        self.big_font = pygame.font.SysFont("monospace", 38, bold=True)
        self.reset()


    def reset(self):
        self.walls = generate_maze(COLS, ROWS)
        self.player = Player(0, 0)
        # One enemy per remaining corner; each runs its own BFS every update.
        self.enemies = [
            Enemy(ROWS-1, COLS-1, (220, 60, 60)),    # bottom-right (red)
            Enemy(0, COLS-1,      (230, 140, 30)),   # top-right (orange)
```

```
            Enemy(ROWS-1, 0,      (160, 60, 200)),   # bottom-left (purple)
        ]
        self.exit_rect = pygame.Rect((COLS//2)*CELL+5, (ROWS//2)*CELL+5, CELL-10, CELL-10)
        self.caught = False
        self.won = False
        # Survival score (1 point per frame alive)
        self.score = 0
        # Difficulty ramp state
        self.start_ticks = pygame.time.get_ticks()
        self.elapsed_ms = 0
        self.tier = 0  # number of speed-ups applied so far
        # Power pellet state
        self._place_pellet()
        self.freeze_frames = 0


    def _place_pellet(self):
        # Any cell except the four corners (player/enemy starts) and the exit cell
        excluded = {(0, 0), (0, COLS-1), (ROWS-1, 0), (ROWS-1, COLS-1), (ROWS//2, COLS//2)}
        cells = [(r, c) for r in range(ROWS) for c in range(COLS) if (r, c) not in excluded]
        r, c = random.choice(cells)
        cx, cy = c*CELL + CELL//2, r*CELL + CELL//2
        self.pellet_center = (cx, cy)
        self.pellet_rect = pygame.Rect(cx - PELLET_RADIUS, cy - PELLET_RADIUS,
                                       PELLET_RADIUS*2, PELLET_RADIUS*2)
        self.pellet_active = True


    def _enemies_maxed(self):
        return all(e.move_interval <= MIN_MOVE_INTERVAL for e in self.enemies)


    def _apply_difficulty_ramp(self):
        # Apply one speed-up per elapsed 15s block (loop in case a frame was long).
        while self.elapsed_ms >= (self.tier + 1) * RAMP_INTERVAL_MS and not
self._enemies_maxed():
            for enemy in self.enemies:
                enemy.move_interval = max(MIN_MOVE_INTERVAL, enemy.move_interval - RAMP_STEP)
            self.tier += 1


    def _update_freeze(self):
        # Count down an active freeze; unfreeze everyone when it runs out.
        if self.freeze_frames > 0:
            self.freeze_frames -= 1
            if self.freeze_frames == 0:
                for enemy in self.enemies:
                    enemy.frozen = False


    def handle_events(self):
        for event in pygame.event.get():
            if event.type == pygame.QUIT: return False
            if event.type == pygame.KEYDOWN and event.key == pygame.K_r: self.reset()
        return True


    def update(self):
        if self.caught or self.won: return  # timer, tier, freeze and score all stop on game over
        self.elapsed_ms = pygame.time.get_ticks() - self.start_ticks
        self._apply_difficulty_ramp()
        keys = pygame.key.get_pressed()
        self.player.move(keys, self.walls, ROWS, COLS)

        # Power pellet: tick any running freeze first, then check for pickup
        self._update_freeze()
        if self.pellet_active and self.player.rect.colliderect(self.pellet_rect):
            self.pellet_active = False
            for enemy in self.enemies:
                enemy.frozen = True
            self.freeze_frames = FREEZE_FRAMES

        for enemy in self.enemies:
            enemy.update(self.walls, self.player, ROWS, COLS)  # no-op while frozen
        if any(self.player.rect.colliderect(e.rect) for e in self.enemies):
            self.caught = True
        elif self.player.rect.colliderect(self.exit_rect):
            self.won = True

        # Survival score: +1 for every frame the player is still alive
        if not (self.caught or self.won):
```

```
            self.score += 1

    def draw(self):
        self.screen.fill((230, 220, 210))
        wc=(50,40,60)
        for r in range(ROWS):
            for c in range(COLS):
                x,y=c*CELL,r*CELL
                w=self.walls[r][c]
                if w[0]: pygame.draw.line(self.screen,wc,(x,y),(x+CELL,y),3)
                if w[1]: pygame.draw.line(self.screen,wc,(x,y+CELL),(x+CELL,y+CELL),3)
                if w[2]: pygame.draw.line(self.screen,wc,(x+CELL,y),(x+CELL,y+CELL),3)
                if w[3]: pygame.draw.line(self.screen,wc,(x,y),(x,y+CELL),3)
        pygame.draw.rect(self.screen,(80,200,80),self.exit_rect,border_radius=4)
        lbl=self.small_font.render("EXIT",True,(20,80,20))
        self.screen.blit(lbl,lbl.get_rect(center=self.exit_rect.center))

        if self.pellet_active:
            pygame.draw.circle(self.screen, PELLET_COLOR, self.pellet_center, PELLET_RADIUS)
            pygame.draw.circle(self.screen, (170, 130, 0), self.pellet_center, PELLET_RADIUS, 2)

        self.player.draw(self.screen)
        for enemy in self.enemies:
            enemy.draw(self.screen)

        # HUD: hint, then time / speed tier (+ freeze), then survival score
        hud=pygame.Rect(0,ROWS*CELL,WIDTH,HUD_H)
        pygame.draw.rect(self.screen,(30,30,50),hud)
        info=self.hud_font.render("Reach EXIT before the enemy catches you!
R=Restart",True,(200,200,200))
        self.screen.blit(info,(8,ROWS*CELL+5))

        secs = self.elapsed_ms // 1000
        tier_label = f"Speed tier {self.tier + 1}" + (" MAX" if self._enemies_maxed() else "")
        interval = self.enemies[0].move_interval
        status = f"Time {secs//60:02d}:{secs%60:02d}   {tier_label}   (enemy moves every
{interval}f)"
        status_color = (240, 120, 100) if self._enemies_maxed() else (160, 220, 160)
        stat=self.small_font.render(status,True,status_color)
        self.screen.blit(stat,(8,ROWS*CELL+27))

        if self.freeze_frames > 0:
            secs_left = (self.freeze_frames + FPS - 1) // FPS  # round up
            fz=self.small_font.render(f"FROZEN {secs_left}s",True,(140,200,255))
            self.screen.blit(fz,(WIDTH-fz.get_width()-8,ROWS*CELL+27))

        surv=self.small_font.render(f"Survived: {self.score // 60}s",True,(255,230,120))
        self.screen.blit(surv,(8,ROWS*CELL+46))

        if self.caught:
            self._overlay("CAUGHT!", (220,60,60))
        elif self.won:
            self._overlay("ESCAPED!", (80,220,80))
        pygame.display.flip()

    def _overlay(self, text, color):
        surf=pygame.Surface((WIDTH,ROWS*CELL),pygame.SRCALPHA)
        surf.fill((0,0,0,140))
        self.screen.blit(surf,(0,0))
        mid=ROWS*CELL//2
        msg=self.big_font.render(text,True,color)
        score_txt=self.font.render(f"Survived: {self.score // 60}s (score
{self.score})",True,(255,230,120))
        sub=self.font.render("Press R to Restart",True,(200,200,200))
        self.screen.blit(msg,(WIDTH//2-msg.get_width()//2,mid-50))
        self.screen.blit(score_txt,(WIDTH//2-score_txt.get_width()//2,mid+5))
        self.screen.blit(sub,(WIDTH//2-sub.get_width()//2,mid+40))

    def run(self):
        running=True
        while running:
            running=self.handle_events()
            self.update()
            self.draw()
            self.clock.tick(FPS)
```

```
        pygame.quit()
```

game/maze.py

```
import random
from collections import deque

CELL = 44

def generate_maze(cols, rows):
    visited = [[False]*cols for _ in range(rows)]
    walls = [[[True,True,True,True] for _ in range(cols)] for _ in range(rows)]
    def neighbors(r,c):
        dirs=[(-1,0,0,1),(1,0,1,0),(0,1,2,3),(0,-1,3,2)]
        res=[]
        for dr,dc,wd,od in dirs:
            nr,nc=r+dr,c+dc
            if 0<=nr<rows and 0<=nc<cols and not visited[nr][nc]:
                res.append((nr,nc,wd,od))
        return res
    stack=[(0,0)]
    visited[0][0]=True
    while stack:
        r,c=stack[-1]
        nbrs=neighbors(r,c)
        if nbrs:
            nr,nc,wd,od=random.choice(nbrs)
            walls[r][c][wd]=False
            walls[nr][nc][od]=False
            visited[nr][nc]=True
            stack.append((nr,nc))
        else:
            stack.pop()
    return walls

def bfs(walls, start, goal, rows, cols):
    """Return next step direction from start toward goal using BFS."""
    sr,sc=start; gr,gc=goal
    queue=deque([(sr,sc,[])])
    visited={(sr,sc)}
    dir_map={0:(-1,0),1:(1,0),2:(0,1),3:(0,-1)}
    opp={0:1,1:0,2:3,3:2}
    while queue:
        r,c,path=queue.popleft()
        if r==gr and c==gc:
            return path[0] if path else None
        for d,(dr,dc) in dir_map.items():
            if not walls[r][c][d]:
                nr,nc=r+dr,c+dc
                if (nr,nc) not in visited and 0<=nr<rows and 0<=nc<cols:
                    visited.add((nr,nc))
                    queue.append((nr,nc,path+[(dr,dc)]))
    return None
```

game/entities.py

```
import pygame
from game.maze import CELL, bfs


SPEED = 2
WALL_T = 3  # wall thickness in pixels (matches the line width used in draw)
FROZEN_TINT = (140, 200, 255)



def wall_rects_for_cell(walls, r, c):
    """Return thin collision rects for every wall present on cell (r, c)."""
    x, y = c * CELL, r * CELL
    t = WALL_T
    h = t // 2  # lines are drawn centred on the cell edge
    rects = []
```

```
    w = walls[r][c]
    if w[0]: rects.append(pygame.Rect(x - h, y - h, CELL + t, t))          # top
    if w[1]: rects.append(pygame.Rect(x - h, y + CELL - h, CELL + t, t))   # bottom
    if w[2]: rects.append(pygame.Rect(x + CELL - h, y - h, t, CELL + t))   # right
    if w[3]: rects.append(pygame.Rect(x - h, y - h, t, CELL + t))          # left
    return rects



class Player:
    def __init__(self, r, c):
        self.r, self.c = r, c
        cx, cy = c*CELL+CELL//2, r*CELL+CELL//2
        self.rect = pygame.Rect(cx-10, cy-10, 20, 20)
        self.color = (60, 120, 220)


    def move(self, keys, walls, rows, cols):
        dx = dy = 0
        if keys[pygame.K_LEFT] or keys[pygame.K_a]: dx = -SPEED
        if keys[pygame.K_RIGHT] or keys[pygame.K_d]: dx = SPEED
        if keys[pygame.K_UP] or keys[pygame.K_w]: dy = -SPEED
        if keys[pygame.K_DOWN] or keys[pygame.K_s]: dy = SPEED
        # Move one axis at a time so the player can slide along walls
        nr = self.rect.move(dx, 0)
        if self._valid(nr, walls, rows, cols): self.rect = nr
        nr = self.rect.move(0, dy)
        if self._valid(nr, walls, rows, cols): self.rect = nr


    def _valid(self, rect, walls, rows, cols):
        # Stay inside the maze area
        if rect.left < 0 or rect.top < 0 or rect.right > cols * CELL or rect.bottom > rows *
CELL:
            return False
        # Check walls of every cell the rect overlaps
        r0, r1 = rect.top // CELL, (rect.bottom - 1) // CELL
        c0, c1 = rect.left // CELL, (rect.right - 1) // CELL
        for r in range(r0, r1 + 1):
            for c in range(c0, c1 + 1):
                for wr in wall_rects_for_cell(walls, r, c):
                    if rect.colliderect(wr):
                        return False
        return True


    def draw(self, screen):
        pygame.draw.ellipse(screen, self.color, self.rect)



class Enemy:
    def __init__(self, r, c, color=(220, 60, 60)):
        self.r, self.c = r, c
        cx, cy = c*CELL+CELL//2, r*CELL+CELL//2
        self.rect = pygame.Rect(cx-12, cy-12, 24, 24)
        self.color = color
        self.timer = 0
        self.move_interval = 20  # frames between cell moves
        self.frozen = False      # set by the power pellet


    def update(self, walls, player, rows, cols):
        if self.frozen:
            return  # frozen enemies do nothing (timer is paused too)
        self.timer += 1
        if self.timer >= self.move_interval:
            self.timer = 0
            pr, pc = player.rect.centery//CELL, player.rect.centerx//CELL
            step = bfs(walls, (self.r, self.c), (pr, pc), rows, cols)
            if step:
                dr, dc = step
                self.r += dr; self.c += dc
                cx, cy = self.c*CELL+CELL//2, self.r*CELL+CELL//2
                self.rect.center = (cx, cy)


    def draw(self, screen):
        if self.frozen:
            # Blend the enemy's own color toward ice blue so enemies stay distinguishable
            body = tuple((a + b) // 2 for a, b in zip(self.color, FROZEN_TINT))
        else:
```

```
            body = self.color
        pygame.draw.rect(screen, body, self.rect, border_radius=5)
        if self.frozen:
            pygame.draw.rect(screen, (225, 245, 255), self.rect, width=2, border_radius=5)
        # eyes
        for ex in [self.rect.x+4, self.rect.x+14]:
            pygame.draw.circle(screen, (255,255,255), (ex, self.rect.y+8), 4)
            pygame.draw.circle(screen, (0,0,0), (ex+1, self.rect.y+8), 2)
```

requirements.txt

```
pygame==2.6.1
```

7. Chat History

Paste the full exported conversation with the vibe coding tool (all prompts and responses, in order). If it is long, paste it here as text or screenshots, and also upload the exported chat PDF to the Lab-4 folder.

Chat link: https://claude.ai/share/fc09bc25-b854-47b8-8679-930d1c4e51e2 (long code listings are omitted here; final code is in Section 6)

You: [Message 1 - fix the broken code] I'm working on a pygame project called Maze Chase: the player escapes a maze while an AI enemy chases using BFS pathfinding. The code is broken. When I run python main.py, this happens: UserWarning: pkg_resources is deprecated as an API. See https://setuptools.pypa.io/en/latest/pkg_resources.html. The pkg_resources package is slated for removal as early as 2025-11-30. Refrain from using this package or pin to Setuptools<81. (from pkg_resources import resource_stream, resource_exists) pygame 2.6.1 (SDL 2.28.4, Python 3.13.7). Hello from the pygame community. https://www.pygame.org/contribute.html. The player can go over the lines and doesnt stay confined within the maze lines. Find all the bugs, briefly explain each one (file + cause), and give me the complete corrected version of every file you changed. [Followed by the starter main.py, game/game_engine.py, game/maze.py and game/entities.py.]

Claude: I found one real bug and a few smaller ones. The pkg_resources message is only a warning, not an error. pygame 2.6.1 imports pkg_resources internally, and your setuptools is new enough to flag it. The game runs fine despite it.

Bugs

- game/entities.py, Player._valid (the main bug): it only checks that the player stays inside the window. It never looks at walls, so the player passes straight through every maze line. It now builds thin wall rectangles, matching the 3px lines drawn in draw(), for each cell the player overlaps and rejects the move if it collides with one.
- main.py, deprecation warning (not a game bug): the warning comes from pygame's own import. You can silence it with a warnings filter set before pygame is imported, or by running pip install "setuptools<81". The filter also hides pygame's greeting line.
- game/game_engine.py, HUD text overflow: the 22pt hint text is about 680px wide in a 572px window, so it gets cut off. It now uses a smaller font.
- game/game_engine.py, "EXIT" label overflow: the 22pt label is wider than the 34px exit box. It now uses a small font and is centered in the box.
- game/game_engine.py, caught and won on the same frame: both flags could be set at once, which draws two overlays. Being caught now takes priority.
- game/entities.py, bfs imported inside Enemy.update: it was re-imported every frame. I moved it to the top of the file.
- game/maze.py is correct. The wall indices in the generator and the BFS direction map agree, so I left it unchanged.

[Claude then gave the complete corrected main.py, game/entities.py and game/game_engine.py.]

One design note, which I left as-is because it isn't a bug: the enemy teleports a full cell every 20 frames instead of gliding. If you want smoother motion, interpolate its rect toward the target cell center each frame.

You: [Task 1 - Multiple enemies] In my maze-chase pygame game, I have one Enemy that uses BFS to chase the player. Add two more Enemy instances starting at different maze corners. Each should independently call bfs() on every update. List them in a self.enemies list and loop over them. Give me the complete updated files.

Claude: I changed two files. game/game_engine.py now creates three enemies in self.enemies, and game/entities.py lets Enemy take an optional color so you can tell them apart. main.py and game/maze.py are unchanged.

What changed in game_engine.py:

- reset() builds self.enemies with enemies at the bottom-right (the original), top-right and bottom-left corners. The player starts at the top-left, so all four corners are used.
- update() loops over self.enemies, so each one runs its own timer and calls bfs() independently.
- The caught check uses any(...) over the list.
- draw() loops over the list to draw each enemy.

[Claude then gave the complete updated game/entities.py and game/game_engine.py.]

With three chasers the game gets much harder, because all of them take the shortest path to you. If it feels too tight, you can raise move_interval on the new enemies (for example 26 and 32) in reset() or Enemy.__init__. All three also move on the same frame since their timers start together, so staggering their starting timer values would make their movement look less synchronized.

You: [Task 2 - Difficulty ramp] Add a difficulty ramp to my maze-chase game. Track elapsed time with pygame.time.get_ticks(). Every 15 seconds, reduce enemy.move_interval by 2 (minimum 5) for all enemies. Show current enemy speed tier in the HUD. Give me the complete updated files.

Claude: Only game/game_engine.py changed. main.py, game/maze.py and game/entities.py are unchanged.

How it works:

- reset() records pygame.time.get_ticks() as the start time, so pressing R restarts both the clock and the difficulty.
- update() computes elapsed time from get_ticks(). It returns early once you're caught or have won, so the timer and speed tier freeze on the end screen.
- Each time another 15 seconds pass, every enemy's move_interval drops by 2, clamped at 5. From the starting 20 frames, the intervals go 20, 18, 16, 14, 12, 10, 8, 6, then 5, which is the cap.
- The HUD is now two lines: the hint on top, and below it the elapsed time, speed tier and current enemy move interval. The tier shows MAX once every enemy has reached the minimum interval.

[Claude then gave the complete game/game_engine.py.]

Things to know:

- With three BFS chasers, the later tiers get very hard. At the minimum interval of 5, each enemy covers a cell about every 83 ms, while the player moves at 2 px per frame (about 120 px/s), so enemies end up much faster than you. If that's too punishing, raise MIN_MOVE_INTERVAL to 8 or 10, or make RAMP_INTERVAL_MS longer.
- The ramp changes each enemy's own move_interval, so it still works if you later give the enemies different starting speeds. The HUD shows the first enemy's interval as the representative value.
- The clock uses wall time from get_ticks(), so it keeps running if you drag or minimize the window.

You: [Task 3 - Power pellet] Add a power pellet to my maze-chase pygame game: a yellow circle somewhere in the maze. When the player rect collides with it, set frozen = True on all enemies and start a 300-frame countdown. While frozen, enemy.update() does nothing. Show a frozen indicator on the enemies. Give me the complete updated files.

Claude: Both game/entities.py and game/game_engine.py changed. main.py and game/maze.py are unchanged.

What changed:

- Enemy: it has a new frozen flag, and update() returns immediately while it's set, so a frozen enemy neither moves nor advances its move timer. When frozen, draw() tints the enemy ice-blue and adds a light outline, so it's obvious at a glance.
- Pellet placement: reset() places one yellow pellet in a random cell. It never uses the four corners (player and enemy starts) or the exit's cell, and every cell in the maze is reachable, so the pellet always is too.
- Pickup: when the player rect touches the pellet, it disappears, every enemy gets frozen = True, and freeze_frames is set to 300.
- Countdown: it ticks down once per frame in update() and unfreezes all enemies when it hits 0. Because it counts frames rather than wall time, it freezes along with everything else on the end screen.
- HUD: a FROZEN 5s countdown appears at the right end of the status line while the effect is active.

[Claude then gave the complete game/entities.py and game/game_engine.py.]

Things to know:

- Touching frozen enemies: a frozen enemy still catches you if you touch it, since you only asked for update() to do nothing. If you'd rather make them harmless while frozen, change the caught check to any(self.player.rect.colliderect(e.rect) for e in self.enemies if not e.frozen).

- Difficulty ramp: it keeps running during a freeze, so enemies can come out of it at a higher speed tier than they went in with.

- Single pellet: there is one pellet per game and it doesn't respawn. If you want a respawning pellet, call self._place_pellet() again after the freeze ends.

- Frame-based countdown: the freeze counts frames, so it lasts exactly 5 seconds at 60 FPS. If the game drops frames, the freeze lasts longer in real time.

You: [Task 4 - Survival score] Add a survival score to maze-chase. Increment self.score by 1 each frame the player is alive. Display it in the HUD as "Survived: Xs" where X is score // 60. On game over, show the final score on the overlay. Give me the complete updated files.

You: [Final review] Please review my final code for bugs or edge cases (e.g. enemies spawning on walls, pellet in unreachable spot, score not resetting on restart).

Claude: I read through the final code but didn't run it. There are no crashes, and your three examples are all fine. I found one real edge case, one gameplay problem and a few smaller things.

Your three cases

- Enemies spawning on walls: this can't happen. Enemies and the player spawn at cell centers. The enemy rect is 24px in a 44px cell, so it sits 10px from each edge, and walls are only 3px thick and centered on the cell edges.

- Pellet in an unreachable spot: this can't happen either. generate_maze is a depth-first spanning tree, so every cell is connected to every other. The pellet only avoids the corners and the exit cell. The corridors leave 41px of clear space and the player is 20px wide, so the player fits through every opening.

- Score not resetting on restart: it does reset. reset() sets score, elapsed_ms, tier, start_ticks and freeze_frames, and it rebuilds the enemies with fresh move_interval and frozen values. It also re-places the pellet.

Real issues

- An enemy can fail to catch you (game_engine.py and entities.py). When an enemy is in your cell, bfs returns None, so it stops at the cell center. A catch needs the rects to overlap, which means the centers must be under 22px apart. If you stand exactly on a cell boundary (an open doorway), you are exactly 22px from the center. Edges that only touch don't count as a collision, so you are safe there indefinitely. It's narrow, but it's an exploitable spot. The fix is to also count "same cell" as caught, in update():

```
    pcell = (self.player.rect.centery // CELL, self.player.rect.centerx // CELL)
    if any(self.player.rect.colliderect(e.rect) or (e.r, e.c) == pcell for e in
self.enemies):
        self.caught = True
```

- The three enemies collapse into one (gameplay). All three run BFS to the same target, so once their paths merge they walk in lockstep on the same cell, and you see one sprite. Starting from different corners only delays this. To keep them distinct, give each a different target (for example, one aims at the player's cell, one at a cell a few steps ahead of the player, one patrols), or block a move into a cell another enemy already occupies.

- The HUD text still says "the enemy" (game_engine.py). There are three now, so change the hint to "Reach EXIT before the enemies catch you! R=Restart".

Minor things

- Mixed timing sources: the speed ramp and the Time readout use wall-clock time, while the score and the freeze count frames. On a slow machine, or while dragging the window, Survived falls behind Time. Freeze ends later than 5 real seconds, and the ramp can ratchet up during a freeze. If you want consistency, drive everything from the frame count.

- Fonts: SysFont("monospace") falls back to a different font on some systems. The status line is about 450px wide, plus the FROZEN label on the right, in a 572px window. That's fine now, but a wider fallback font could make them collide.

- Frozen enemies are still lethal: this is the design choice from before. It is easy to flip if you want a frozen enemy to be harmless.

I can apply the first three fixes and give you the complete files if you want.

8. Final Repo Structure (Lab-4 folder)

```
SELABS/
  Lab-4/
    before.mp4, after.mp4
    code/ (main.py, game/, requirements.txt)
    PES2UG24CS571.pdf
```